Make groups of three Students each, and every group work on the following topics:

Topic 1: Graph concepts:

- Introduction to Graphs **(Done)**
    - Graphs as collections of nodes and edges
    - Real World scenarios where graphs model relationships
- Types of Graphs **(Done)**
- Graph Representation
    - Adjacency Matrix (Pros and Cons)
    - Adjacency List (Space and Time efficiency)
- Graph Traversal
    - Depth-First Search (DFS)
    - Breadth-First Search (BFS)
- Application of Graphs
    - Shortest Path Algorithms
    - Minimum Spanning Trees
    - Topological Sorting

Implementing Graphs in C++:

- Graph Implementation
    - Node and Edge classes for graph representation
    - Storing graphs using adjacency lists
- Utilizing Standard Libraries
    - C++ STL for data structures
    - Use STL containers for tree and graph operations

Topic 2: Trees Concepts.

- Introduction to Trees
    - Trees as Hierarchical data structures
    - Trees in various applications
- Types of Trees
    - Binary Trees
    - Binary Search Trees (BST)
    - Balanced Tree Concepts

- B-Trees

- Tree Traversal Techniques
    - In order, Pre-Order, and Post-Order Traversals
    - Level-Order traversal for breadth-first exploration
- Tree Operations and Algorithms
    - Insertion and Deletion of Nodes in BST
    - Balancing Operations in Self-Balancing Trees
    - Search and Modifications Algorithms
- **Implementing Trees in C++:**
- Tree Implementation
    - Use of Classes / Structs for tree nodes
    - Recursive methods for insertion, deletion, and traversal

Topic 3: Map Data Structures and Algorithms

- Introduction to map data structures (associative arrays)
    - Defining map data structures and their role in data storage
    - The concept of associative arrays
    - the importance of efficient key-value storage
- Key-value pairs and their role in data storage.
    - The concept of key-value pairs
    - Role of key-value pairs in data organization and retrieval
    - Relationship between keys and values
- Map operations: insertion, deletion, retrieval.
    - Inserting key-value pairs into a map.
    - Techniques for deleting entries from a map.
    - Retrieving values based on given keys.
    - Handling cases of duplicate keys and updating values.
- Exploring C++ implementations: std::map and std::unordered_map
    - Introduction to std::map and std::unordered_map in C++ STL
    - The differences between ordered and unordered maps
    - The usage of map implementations in C++
- Application of maps in data association and problem solving
    - Using maps for dictionary and spell-check applications.

- Maps in implementing frequency counters.
- Using maps for efficient data association.

Topic 4: Problem Solving and Optimization of Graphs and Trees with C++

- Constructing Binary Trees:
    - Building binary trees and BSTs using C++
    - Techniques for insertion, deletion, and traversal
- Creating and Navigating Graphs:
    - Graph creation and defining edge associations.
    - Algorithms to traverse and explore graphs.
- Family Tree Representation:
    - Applying binary trees to model family genealogy
    - Precise traversal for family relationships exploration
- Social Network Analysis:
    - graph representation of social connections
    - algorithms to discover mutual friends.
- Handling Imbalanced Trees:
    - Balancing techniques for BST
    - Rotations and rebalancing operations
- Graph Algorithms Optimization:
    - Optimize efficiency in graph traversal.
    - Strategies for faster shortest path calculations

Topic 5: Introduction to Hash Tables

- Hash Tables Description
    - Overview of hash tables as efficient data storage structures
    - Importance of hash tables in solving data retrieval challenges
    - Role of hash functions in hash table design
- Memory Allocation and Hash Tables
    - Principles of memory allocation in hash table implementation
    - Allocating memory for hash table components

        - Analyzing memory allocation's impact on hash table performance

- **Hash Function Principles**

    - Hash Function Basics

        - Exploring the fundamental concepts of hash functions
        - Understanding how hash functions generate indices
        - Relationship between data and hash table indices

    - Hash Function Design

        - Factors influencing hash function design.
        - Principles for creating effective hash functions.
        - Hash function collision avoidance strategies.

- **Data Retrieval in Hash Tables**

    - Data Retrieval Concepts:

        - Importance of efficient data retrieval in hash tables.
        - Applying hash table data retrieval to real-world problems.
        - Techniques for enhancing data retrieval performance.
    - Problem Context and Hash Table Retrieval:

        - Adapting hash table data retrieval to specific problem contexts.
        - Matching retrieval methods with data retrieval requirements.
        - Ensuring data integrity during hash table retrieval.

- **Generating Indices with Hash Functions**

    - Indices Generation and Storage:

        - Implementing hash functions to generate indices.
        - Mapping data to hash table indices.
        - Handling index distribution for optimized data storage.

    - Hashing for Data Storage:

        - Correlation between hash functions and data storage.
        - Mapping data to hash table indices using hash functions.
        - Strategies for handling hash collisions.

- **Collision Resolution Strategies**

    - Introduction to Collision Resolution:

        - Introduction to collision resolution in hash tables.
        - Challenges posed by hash collisions.
        - Importance of suitable collision resolution techniques.

    - Collision Resolution Techniques:

        - Detailed exploration of collision resolution methods.
        - Implementing open addressing for collision resolution.
        - Chaining technique for resolving hash collisions.

    - Open Addressing:

        - Understanding open addressing as a collision resolution technique.
        - Implementing linear probing, quadratic probing, and double hashing.
        - Exploring scenarios where open addressing is effective and its limitations.

    - Chaining:

        - Understanding chaining as a collision resolution technique.
        - Implementing linked lists for collision resolution.
        - Exploring scenarios where chaining is effective and its limitations.

    - Efficient Hash Table Design:

        - Strategies for designing hash functions that distribute data uniformly.
        - Analyzing the impact of hash function quality on hash table performance.
        - Techniques to minimize collisions and maximize retrieval efficiency.

    - Appropriate Implementation of Collision Resolution:

        - Aligning collision resolution with chosen hash table design.
        - Analyzing collision resolution's impact on data retrieval.
        - Identifying scenarios where specific collision resolution techniques excel

- **Implementation of Hash Tables:**

    - Designing and Implementing Hash Tables:

        - Hash table design based on hash function principles.
        - Implementation of hash table components.

        - Application of memory allocation standards to hash table design.

    - Hash Table Data Retrieval:
        - Practical exercises for retrieving data from hash tables.
        - Adapting data retrieval techniques to diverse problem contexts.
        - Analyzing data retrieval efficiency using hash tables.

    - Indices Generation and Collision Handling:
        - Implementing hash functions and generating indices in a lab setting.
        - Implementing collision resolution strategies in hash tables.
        - Analyzing index distribution patterns and addressing collisions.

- **Real-World Applications of Hash Tables**

    - Application of Hash Tables for Data Management:
        - Implementing hash tables to solve real-world problems.
        - Hash tables' role in efficient data storage and retrieval.
        - Evaluating hash tables' advantages in specific scenarios.

    - Data Integrity and Performance Optimization:
        - Optimizing data storage using hash tables.
        - Real-world applications of hash tables in managing data.
        - Comparing hash table performance with other data structures.

NB:

1. **Please start by the first three topics and make sure you are done with them by Wednesday, 15th June, 2024.**
2. **Submit both word and ppt files of your work by the indicated deadline, and submit all of the in one zipped file to the email address hhatangimbabazi@rca.ac.rw**